FocusGuard- Detailed design

1.Introduction

FocusGuard is a local, webcam-based focus monitoring system that helps students maintain concentration during self-directed learning at home. The system analyzes real-time visual cues such as **gaze direction**, **phone usage**, and **absence from the camera frame** to detect meaningful loss of focus. When distraction persists beyond a configurable duration, FocusGuard delivers a **gentle alert** and logs the event for later review.

To preserve privacy and reduce latency, **all video analysis runs on device**. Only **metadata** (scores, states, timestamps, flags) is stored locally, not raw video frames.

---

2. Scope and Requirements

2.1 Stakeholders

- **Student (Primary user)** – starts/stops sessions, views live focus feedback, receives alerts, reviews history, configures thresholds.
- **Instructor/Advisor (Optional reviewer)** – evaluates the system using demo outputs, design diagrams, and reports (not a runtime user).
- **System Owner (Local device admin)** – manages installation, camera selection, privacy mode, storage.

2.2 Functional Requirements (summary)

FR-01 The system shall allow a student to start a focus monitoring session.
FR-02 The system shall capture video frames from a standard webcam during an active session.
FR-03 The system shall detect face presence in the camera frame.
FR-04 The system shall estimate gaze direction and identify sustained “looking away” behavior.
FR-05 The system shall detect prolonged absence from the camera frame.
FR-06 The system shall compute a real-time focus score and classify the focus state (Focused / Distracted / Away).
FR-07 The system shall generate gentle alerts when loss of focus persists beyond a configurable threshold.
FR-08 The system shall stop a focus session and generate a basic session summary.

2.3 Non-Functional Requirements

Privacy:

The system shall process all video data locally on the user’s device. Raw video frames shall not be stored or transmitted to external services.

Performance:

The system shall support near real time focus monitoring on a typical home computer by using lightweight computer vision models and optional frame sampling.

Reliability:

The system shall maintain consistent focus state transitions and handle session start and stop operations without data corruption. Camera access failures shall be handled gracefully.

Usability:

The system shall provide a simple and intuitive user interface with minimal interruption to the learning process. Focus alerts shall clearly indicate the reason for loss of concentration.

Maintainability:

The system shall be designed using modular components with clearly defined interfaces to allow future replacement or extension of detection modules.

---

3. Architecture

3.1 Architecture Style

A **local client–server style architecture**:

- **GUI** (dashboard + history + settings)
- **Backend API** (session control + logic + persistence)
- **Local DB** (sessions/events/settings)
- **Real-time channel** (WebSocket) for live updates and alerts
- **Vision pipeline** (MediaPipe/OpenCV + detector modules)

3.2 High-Level Components

GUI (Student Dashboard)

- Live state (FOCUSED/DISTRACTED/AWAY), score, reason flags
- Start/Stop session
- Settings + Calibration
- History + analytics

Backend API

- Business logic, session management, validation, alert rules
- Exposes REST endpoints and WebSocket events

Vision Pipeline

- Frame capture + preprocessing
- Face landmarks, gaze estimation

- Pose/skeleton analysis
- Phone detection

Focus Engine

- Combines detector outputs into a focus score and focus state
- Uses time-based persistence to avoid false positives

Alert Manager

- Uses thresholds + cooldown to trigger gentle alerts

Local Database

- Stores users (optional), settings, sessions, events, summaries

3.3 Deployment View

Single-device deployment:

- UI and backend run locally (localhost)
- Database is local SQLite file
- Optional packaging as a desktop executable
- No internet required for core functionality

---

4. Data Design

4.1 Core Entities

UserProfile
userId, displayName, createdAt

Settings
userId, gazeThresholdSec, phoneThresholdSec, postureThresholdSec, absenceThresholdSec, alertCooldownSec, frameSampleRate

CalibrationProfile (optional)
userId, baselineGazeCenter, baselinePoseMetrics, createdAt

Session
sessionId, userId, startTime, endTime, avgScore, focusedPercent, distractedPercent, awayPercent

Event (high frequency)
eventId, sessionId, ts, score, state(FOCUSED/DISTRACTED/AWAY), reasonFlags(JSON), gazeYaw, gazePitch, phoneConf(optional)

Alert
alertId, sessionId, ts, type(GAZE/PHONE/POSTURE/ABSENCE), severity(INFO/WARN), triggeredAt, resolvedAt(optional)

4.2 Data Integrity Rules

Session finalization consistency:
The system shall ensure that stopping a session results in a single, consistent session summary, even if the stop operation is triggered more than once.

Stable focus state transitions:
During a continuous distraction episode, the system shall avoid rapid oscillation between focus states by applying time-based persistence rules.

No raw video persistence:
Raw video frames shall not be stored. Only derived metadata (e.g., timestamps, scores, states, and flags) may be persisted.

Ordered event timestamps:
Events within a session shall be recorded with monotonically increasing timestamps to support correct timeline reconstruction and visualization.

---

5. API Specification

The system exposes REST endpoints for session control, configuration, and history access, and uses WebSocket events to deliver real-time focus updates and alert notifications to the GUI.

5.1 Session Control

POST /sessions/start
Starts a focus monitoring session.

- **Request:**
  { userId, configOverrides? }
- **Response:**
  { sessionId }

POST /sessions/{sessionId}/stop
Stops an active session and finalizes the session summary.

- **Response:**
  { summary }

GET /sessions/{sessionId}/summary
Returns aggregated metrics and summary information for a completed session.

5.2 Settings and Calibration

GET /users/{userId}/settings
Retrieves the current configuration settings for the user.

PUT /users/{userId}/settings
Updates user-specific configuration.

- **Request:**
  { thresholds, sampleRate, alertType, cooldown }

POST /users/{userId}/calibration/start *(Optional)*
Initiates an optional calibration process.

POST /users/{userId}/calibration/save *(Optional)*
Stores calibration results for the user.

5.3 History

GET /users/{userId}/sessions?from=&to=
Returns a list of sessions within a specified time range.

GET /sessions/{sessionId}/events?limit=&fromTs=&toTs=
Returns focus events recorded during a session, optionally filtered by time range and limit.

5.4 WebSocket Events (Real-Time)

focus_update

- **Payload:**
  { sessionId, timestamp, score, state, reasonFlags }

alert_triggered

- **Payload:**
  { sessionId, timestamp, type, message }

session_started / session_stopped

- **Payload:**
  { sessionId, timestamp }

6. Key Workflows

6.1 Student Starts a Session

1. The student clicks the “Start” button in the GUI.
2. The GUI sends a request to POST /sessions/start.
3. The backend initializes the session and prepares the computer-vision pipeline.
4. The backend emits a session_started event.

5. The GUI begins receiving real-time focus_update events.

6.2 Real-Time Monitoring Loop

1. The system captures a video frame (or every Nth frame, depending on sampling configuration).
2. The frame is preprocessed (e.g., resizing and normalization).
3. Detection modules are executed (face presence, gaze estimation, posture analysis, and optional phone detection).
4. The focus engine updates internal counters and computes the current focus score and focus state.
5. A focus event record is stored.
6. A focus_update event is emitted to the GUI.
7. If distraction conditions persist and the alert cooldown has passed, an alert_triggered event is emitted.

6.3 Stop Session and Summary

1. The student clicks the “Stop” button in the GUI.
2. The backend stops frame processing and aggregates session statistics.
3. The session summary is stored.
4. The GUI displays the session results and a timeline visualization.

7. Algorithms

7.1 Focus State Classification (Time-Based Persistence)

Plain English Description

1. Each detection module produces a binary or categorical signal for every processed frame, such as:
   lookAway, phoneDetected, badPosture, and faceAbsent.
2. The system maintains a duration counter for each signal, representing how long the condition has persisted.
3. The focus state is determined as follows:
   - If faceAbsentCounter ≥ T_absence, the focus state is classified as AWAY.
   - Else, if any of the distraction counters (gaze, phone, or posture) exceed their respective thresholds, the state is classified as DISTRACTED.
   - Otherwise, the state is classified as FOCUSED.

8.User Interface Design (Screens)

8.1 Live Dashboard

The live dashboard is the primary interaction screen used during a focus session. It provides a concise, real-time summary of the user's attention state and system feedback.

- Focus state badge indicating FOCUSED / DISTRACTED / AWAY
- Focus score gauge (e.g., 0–100)
- Visual indicators explaining detected distractions:
    - gaze deviation
    - phone detected
    - posture deviation
    - absence from camera
- Session timer displaying elapsed time
- Start / Stop session controls
- Small, non-intrusive alert banner (e.g., *"Phone detected – please refocus"*)

8.2 History and Analytics

This screen allows the user to review past focus sessions and overall trends.

- List of completed sessions (date, duration, average score)
- Timeline visualization of focus score over time
- Summary statistics, including:
    - Percentage of time spent Focused / Distracted / Away
    - Count of alerts grouped by type

8.3 Settings

The settings screen enables basic configuration of system behavior.

- Threshold sliders defining alert sensitivity (in seconds)
- Alert modality selection (sound, notification, or both)
- Performance configuration options, such as frame sampling rate

9. Error Handling and Validation

- Camera unavailable or permission denied:
  The system shall display an appropriate error message and terminate the session gracefully.
- Temporary face detection loss:
  Short-term loss of face detection shall not immediately trigger alerts; time-based persistence rules shall be applied to avoid false positives.

- Detector failure:
  If a detection module fails, the system shall continue operating using a reduced set of available features (e.g., face presence only).
- Alert rate control:
  A cooldown mechanism shall be applied to prevent repeated or excessive alert notifications.
- Session finalization consistency:
  Stopping a session shall result in a single, consistent session summary, even if the stop action is triggered more than once.

10.Security and Privacy

- Local-only processing:
  All video analysis shall be performed locally on the user's device. No video data shall be transmitted to external services.
- Data minimization:
  The system shall store only derived metadata (e.g., focus scores, states, and reason flags). Raw images or video frames shall not be persisted.
- Optional minimal logging mode:
  The system may support an optional mode in which only aggregated session summaries are stored, reducing the amount of retained data.
- User profiles and authentication (optional):
  If local user profiles are supported, authentication data shall be stored securely (e.g., using hashed credentials). For demonstration purposes, authentication may be omitted.

---

11. Testing

- Unit Testing:
  Unit tests shall be designed to validate core logic components, including focus state transitions, duration counters, alert cooldown behavior, and the focus scoring function.
- Integration Testing:
  Integration tests shall verify end-to-end interactions between system components, including session start and stop flows, event logging, and the structure of WebSocket updates sent to the GUI.
- Manual Testing:
  Manual tests shall be performed to evaluate system behavior under real-world conditions, such as variations in lighting, different phone orientations, and user appearance changes (e.g., glasses, head movement).

.

---

12. Deployment and Operations

- Local development environment:
  During development, the GUI and backend services shall run locally on the user's machine (e.g., via localhost communication), using a local SQLite database file for persistence.
- Logging:
  The system shall generate structured log entries for key lifecycle events, such as session start and stop actions and alert triggers, while avoiding logging sensitive visual data.
- Health monitoring:
  The system may expose a lightweight health endpoint (e.g., GET /health) to report basic readiness status, including camera availability and database accessibility.
- Optional packaging:
  The application may be packaged as a standalone desktop executable for demonstration or distribution purposes.

Appendix A: UML Diagrams

A1. Communication / Component Diagram

```
@startuml

component "GUI (Dashboard)" as UI

component "Backend API" as API

database "Local DB (SQLite)" as DB

component "WebSocket" as RT

component "Vision Pipeline" as CV

component "Alert Manager" as AM


UI --> API : REST/JSON

UI --> RT : WebSocket events

API --> CV : process frames / extract features

API --> DB : read/write sessions & events

API --> AM : evaluate thresholds + cooldown

AM --> RT : alert_triggered

@enduml
```

A2. Sequence Diagram (Live Session)

```
@startuml

actor Student as S
```

```
participant "GUI" as UI
participant "Backend API" as API
participant "Vision Pipeline" as CV
database "SQLite" as DB
participant "WebSocket" as WS

S -> UI : Start Session
UI -> API : POST /sessions/start
API -> DB : create session
API -> WS : session_started

loop every N frames
  API -> CV : processFrame()
  CV --> API : {score, state, flags}
  API -> DB : insert event
  API -> WS : focus_update
  alt alert required
    API -> WS : alert_triggered
  end
end

S -> UI : Stop Session
UI -> API : POST /sessions/{id}/stop
API -> DB : save summary
API -> WS : session_stopped
@enduml
```

A3. Activity Diagram (Focus Detection)

```
@startuml
start
:Start Session;
```

```
:Capture frame (sampled);
:Preprocess frame;
:Detect face + landmarks;
if (Face present?) then (yes)
  :Estimate gaze;
  :Estimate pose/posture;
  :Detect phone;
  :Update counters;
  :Compute score + state;
else (no)
  :Update absence counter;
  :Set state = AWAY if threshold met;
endif
:Log event;
if (Alert conditions met?) then (yes)
  :Trigger gentle alert;
endif
repeat while (Session active?)
:Stop Session;
:Aggregate metrics + save summary;
stop
@enduml
```

A4. UML Class Diagram

This class diagram presents the main software entities of the FocusGuard system and their responsibilities.
It illustrates how session management, vision processing, focus evaluation, alert handling, and data persistence are separated into modular components with clear relationships.

```
@startuml
class Session {
  sessionId
  startTime
  endTime
```

```
  avgScore
  focusedPercent
  distractedPercent
  awayPercent
  + finalizeSession()
}

class Event {
  eventId
  timestamp
  score
  state
  reasonFlags
}

class Settings {
  gazeThresholdSec
  phoneThresholdSec
  postureThresholdSec
  absenceThresholdSec
  alertCooldownSec
  frameSampleRate
}

class VisionPipeline {
  + captureFrame()
  + preprocessFrame()
  + extractVisualSignals()
}

class FocusEngine {
  + computeFocusScore()
```

```
  + determineFocusState()
  + updateCounters()
}

class AlertManager {
  + checkAlertConditions()
  + triggerAlert()
}

Session "1" --> "*" Event : contains
Session --> Settings : uses
FocusEngine --> VisionPipeline : processes
FocusEngine --> AlertManager : notifies
Session --> FocusEngine : controlled by
@enduml
```

Technology Stack (Confirmed)

- Backend: Python with FastAPI
- Computer Vision: MediaPipe and OpenCV
- Database: SQLite
- Real-Time Communication: WebSocket
- User Interface: React (web-based) or PyQt (desktop-based)
- Deployment Model: Local device execution with a privacy-first approach

Coding Standards

- Modular, feature-based folder structure (e.g., api/, vision/, focus/, alerts/, storage/, ui/)
- Consistent naming conventions: PascalCase for classes, camelCase for functions and variables
- Centralized error handling with a consistent JSON-based error response schema
- Logging practices that avoid storing sensitive visual or personal data